COMP.SE.110 SOFTWARE DESIGN, 2026–2027

City Tour Planner

Prototype and Initial Design

Submission: Prototype and initial design

Team: Varun Ganesh Sondur, Pham Thanh, Georgios Malezoglou, Abdullah

1 Project Overview

City Tour Planner helps users plan a city visit around their available time and the weather. The user selects a city, date, start time, and tour duration. Our application combines information from external data sources and applies its own planning logic to suggest an ordered set of places to visit. The intended results are a map, an ordered list of stops, weather information, and relevant road traffic announcements when available.

We designed the application to support multiple cities rather than a single location. We used Helsinki as our initial location for investigating candidate APIs and understanding the available data. Our map and city-selection design allows us to extend the application to other cities using suitable data providers (Section 2.3).

1.1 Main user workflow

The user selects a city, date, start time, and tour duration. The application retrieves relevant weather, place, and traffic information, and we process the available data and apply our tour-planning rules. The planning logic selects and orders suitable places within the requested duration. The application then displays the itinerary, map, weather visualization, and available traffic information, and the user can show or hide data layers in the results view. Section 3.3 describes the technical request flow.

1.2 Initial scope

We intend the initial version to support tour planning for multiple cities using appropriate data providers, based on the selected date, start time, and duration. It filters and orders places with application-side logic, with weather-aware recommendations when suitable forecast data is available, and presents a map, an ordered list of suggested stops, weather visualization, and optional traffic information. It gives clear feedback when an external data source is unavailable.

Booking, payments, user accounts, live GPS tracking, and guaranteed optimal pedestrian routing are outside our initial scope. We will present walking times, train information, and precipitation probabilities as factual results only when a suitable data source supports them, and we will distinguish estimates from verified measurements.

2 Data Sources and API Research

2.1 API Research Runner

Before designing our integrations, we developed a separate *API Research Runner* to test candidate endpoints. It records HTTP status codes, response times, content types, response previews, and errors in a JSON report. The endpoints we tested returned HTTP 200 during our initial research. This confirms that the requests succeeded at the HTTP level; it does not confirm that every field needed by our planning logic is available. The Runner is a development utility and not a production component of City Tour Planner.

2.2 Tested data sources

Table 1 summarizes the three sources we investigated and what each test does and does not establish.

2.3 Place data for multiple cities

We designed the application around a *selected city* and a provider integration layer instead of hard-coding Helsinki into the planning logic. The selected city determines which place-data provider we use, based on that provider's coverage. For cities outside the Service Map's coverage, our design allows us to integrate alternative providers that supply relevant place

Table 1. Data sources investigated with the API Research Runner.

Source	Tested response	Planned use and limits
FMI Open Data (weather)	XML	Forecast measurements and timestamps for the weather chart and, where the data supports it, weather-aware place selection. We interpret each measurement by its actual unit and time coverage and do not assume that precipitation probability or any other field is available unless it is present in the retrieved data.
Digitraffic (road traffic)	JSON	Road traffic announcements as additional context for a tour. The tested endpoint provides road announcements only; it does not establish train departure times or delays. We treat rail information as outside the current integration unless we introduce and test a suitable additional source.
Helsinki Service Map [1, 2] (places), `/unit/` endpoint	JSON	Place and service records. The data also contains general city services, so we filter records by category and relevant fields instead of treating every unit as a tourist attraction. Coverage is Helsinki and other municipalities in the Helsinki metropolitan area.

Table 2. Planned technology stack.

Area	Technology	Intended use
Backend	Java 22, Spring Boot 3.5.6	Application workflow and tour-planning logic
Build, configuration	Maven; YAML, Jackson	Dependency management and builds; configuration and JSON processing
API communication	Java HTTP Client or Spring HTTP support	External API requests
Data providers	FMI Open Data, Digitraffic, Helsinki Service Map and additional providers	Weather, road traffic, and place data for supported cities
Frontend	React with Vite	Planning form and results interface
Map	Leaflet with OpenStreetMap tiles (proposed)	Places and suggested stop sequence
Charts	Recharts or Chart.js	Weather visualization
Frontend–backend	REST API, JSON, browser `fetch`	Communication with the Java backend
UI prototype	Figma	Screen designs and prototype interactions

data, without redesigning the workflow. Our API tests so far used Helsinki as the starting location; additional city providers are part of our planned implementation.

2.4 Combining the data

The external APIs supply raw information, and our application turns it into a proposed itinerary. We (1) identify the data sources available for the selected city, (2) retrieve the relevant weather, place, and traffic information, (3) normalize provider-specific responses into application-level models, (4) filter places according to our planning rules, and (5) select and order stops within the requested duration. We keep provider-specific processing separate from the planning logic, distinguish real provider data from estimates, and handle missing information explicitly instead of inventing values.

3 Technologies and Architecture

3.1 Planned technologies

Table 2 lists our main technology direction. We will select the final HTTP integration and charting libraries according to the needs of the implementation.

3.2 Main components

We separate the frontend, application workflow, tour-planning logic, and external API integrations (Table 3).

3.3 Component interaction and request flow

Diagram 1 shows the component structure we propose. It describes the architecture we intend to implement, not a claim that every component is already complete. The numbered steps below describe the request flow.

Table 3. Main software components and their responsibilities.

Component	Responsibility
React frontend	Collects the city, date, start time, and duration; displays the loading state, itinerary, map, weather chart, and traffic information when available; provides controls for showing and hiding data layers.
Application layer	Receives and validates tour requests, coordinates data retrieval and planning, and returns the generated result to the frontend.
Tour-planning logic	Filters places using the available data and user selections, applies weather-aware rules where supported, and selects and orders stops within the requested duration.
Provider interfaces and adapters	Isolate external API requests and convert provider-specific responses into application-level models.
External providers	Initially FMI (weather), Helsinki Service Map and additional providers (places), and Digitraffic (road traffic announcements).

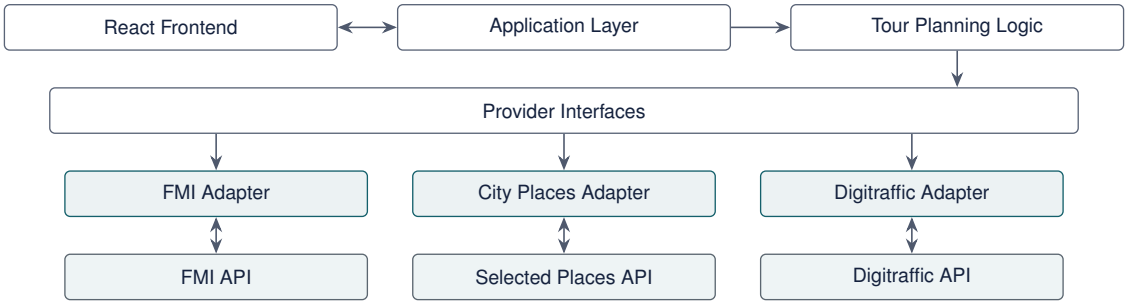


Diagram 1. Component interaction. Single-headed arrows show the direction of control; double-headed arrows show request and response exchanges.

1. The user submits a tour request through our frontend.
2. The backend validates the request and identifies the data providers suitable for the selected city.
3. Our integration layer retrieves the relevant weather, place, and traffic data.
4. Provider adapters normalize the responses into application-level models.
5. Our planning logic filters and orders candidate places.
6. The backend returns the itinerary and visualization data.
7. The frontend displays the results and any relevant data-availability messages.

We separate these layers so that the planning logic does not depend on the response formats of individual providers and so that we can add or replace a data source as city support grows. The workflow also tolerates partial data availability: if one provider fails, the application can explain the limitation and use other suitable data where possible, instead of silently treating missing information as valid.

4 Design Approach

4.1 Adapter pattern

We chose the Adapter pattern as our main approach for integrating external providers that have different endpoints and response formats. For example, we can use `FmiWeatherAdapter`, `ServiceMapPlacesAdapter`, other city-specific place adapters, and `DigitrafficAdapter`. Each adapter converts the corresponding external response into our application's data models. This keeps provider-specific details out of the tour-planning logic and supports our multi-city design, because we can add or replace a place provider without rewriting the planning workflow. Related constraints carry into the implementation: we select providers per city instead of assuming one source covers every location, we filter general service records into relevant places, we use forecast values according to their actual units, timestamps, and coverage, and we treat road traffic announcements separately from train schedules and delays.

4.2 Strategy pattern

We may need different selection rules for general, indoor-focused, and outdoor-focused tours. If these rules become sufficiently different, we can represent them through a common strategy interface. For the initial version, we intend to keep the planning logic proportionate to the requirements and to introduce separate strategy implementations only where they improve the design.

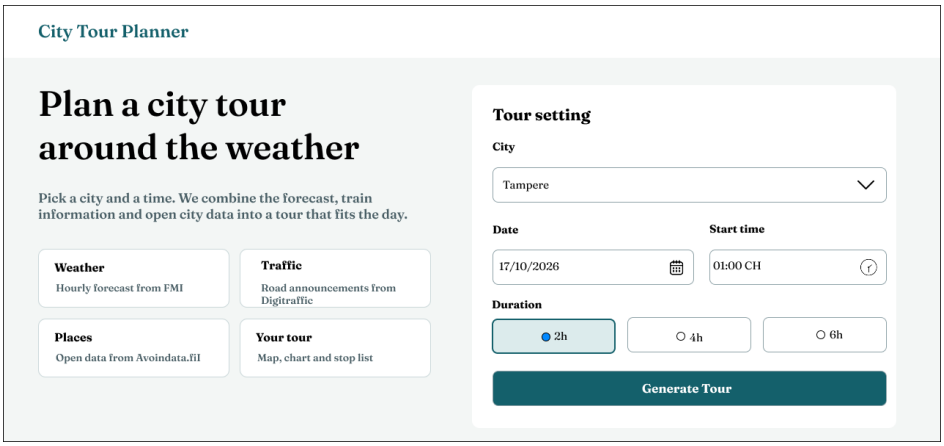


Figure 1. City Tour Planner planning screen.

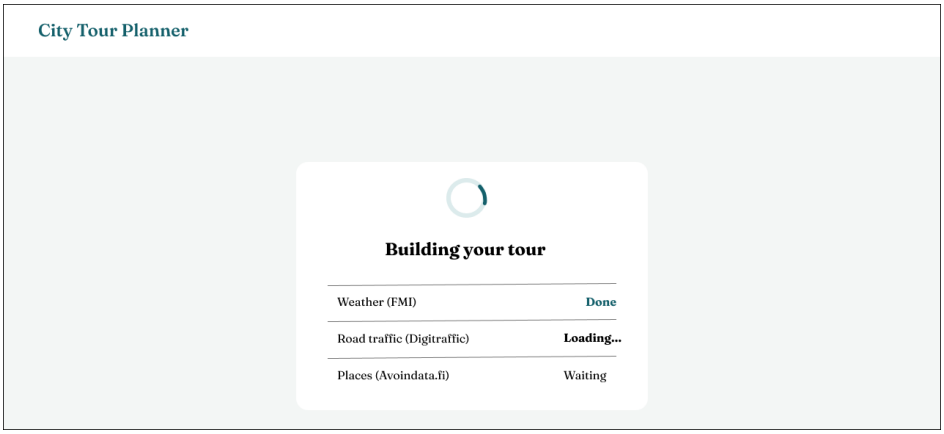


Figure 2. Loading state showing progress across data sources.

5 UI Prototype

Our Figma prototype [3] demonstrates the intended user journey from entering tour preferences to reviewing the suggested itinerary. The figures below are screenshots exported from the prototype; we cropped only empty space at the bottom of each screen. The weather values (70% chance of rain, 6 ℃, 4 m/s wind), stop times, walking times, route lines, and traffic announcements in these screens are illustrative prototype content. The prototype marks the map as a placeholder for Leaflet and the traffic rows as sample text. We do not present any of these values as live or verified data.

5.1 Planning screen

The planning screen (Figure 1) provides a city selector, date, start time, tour-duration options (2 h, 4 h, 6 h), and a *Generate Tour* action. The panel on the left introduces the main features: weather information, traffic information, places, and the proposed tour. The selected city determines which data providers the backend uses, which reflects our decision to design the application for multiple cities. The introductory text on this screen mentions train information; as described in Section 2.3 and Table 1, the Digitraffic endpoint we tested covers road announcements only, and rail information is outside our current integration.

5.2 Loading screen

The loading screen (Figure 2) communicates progress while data is retrieved. It lists weather, road traffic, and places as separate retrieval steps with their own status (*Done*, *Loading*, *Waiting*), so users can see what the application is processing. In the implementation, these indicators will reflect actual request states, and we intend to handle failed or delayed requests without leaving the user waiting indefinitely.

5.3 Results screen

The results screen (Figure 3) brings the outputs together: a map with numbered stops and a suggested visit sequence, a stop list with names, categories, and planned times, an hourly weather chart with a tour-window indicator, and a *Why this*

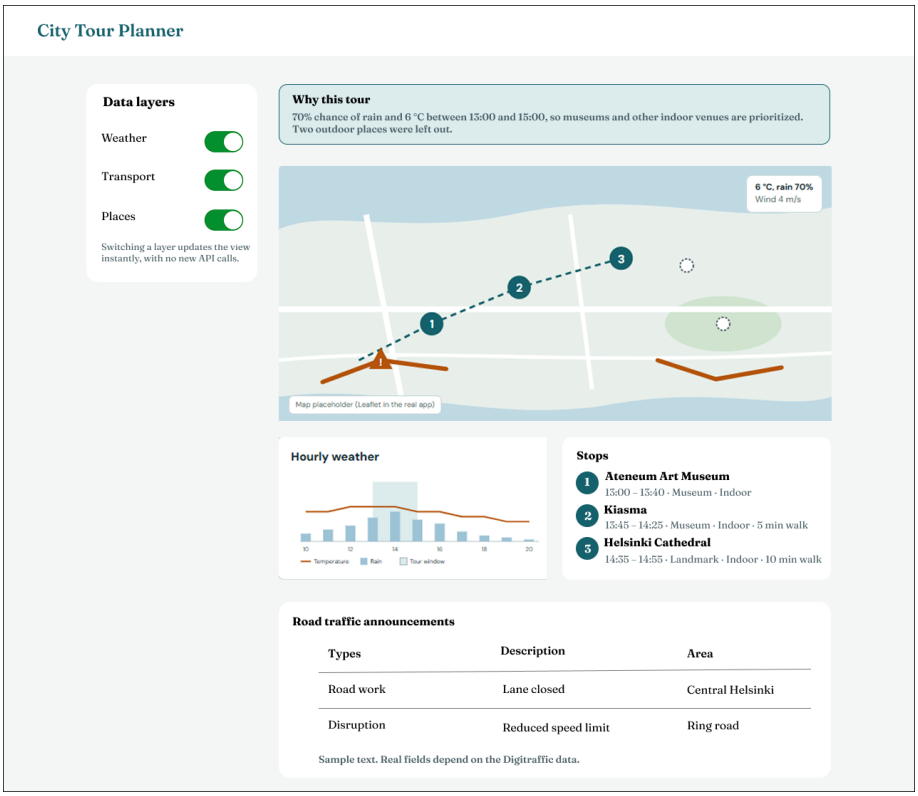


Figure 3. Results screen with the map, weather information, and place layers.

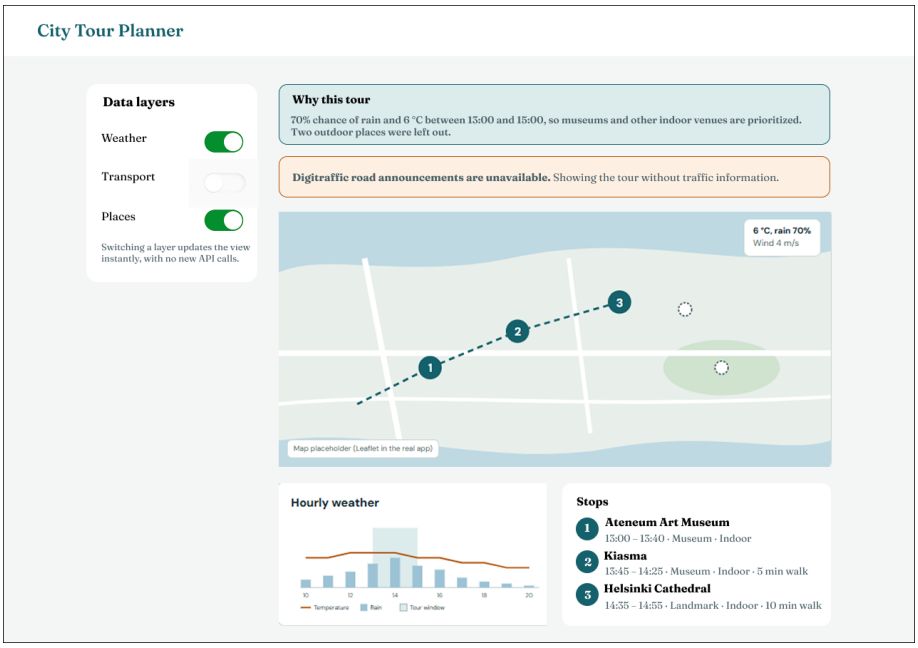


Figure 4. Results screen demonstrating unavailable traffic information.

tour panel that explains how the weather influenced the suggested tour. Road traffic announcements appear in a separate table when available. The *Data layers* panel lets the user switch the weather, transport, and places layers on and off; the prototype specifies that switching a layer updates the view instantly, without new API calls.

5.4 Unavailable-traffic state

We also designed a results state in which traffic announcements are unavailable (Figure 4). A notice states that the Digitraffic road announcements are unavailable and that the tour is shown without traffic information, and the transport layer is switched off. This demonstrates how the application communicates a missing data source instead of silently presenting unavailable information as valid, a behavior that the standard results screen does not show.

6 Team Contributions and AI Usage

6.1 Team contributions

We have provisionally divided the work by area so that we can develop the application in parallel while coordinating integration (Table 4). We will adjust responsibilities when needed, and all members review the overall design.

Table 4. Provisional allocation of responsibilities.

Member	Main area	Contribution and planned responsibilities
Varun Ganesh Sondur	Backend architecture and API integration	Built the API Research Runner, investigated external API feasibility, and contributed to project planning and architecture. Will focus on provider adapters, backend integration, and tour-planning logic.
Pham Thanh	Frontend and UI	Designed the Figma prototype and contributed the screen content based on API findings. Will focus on the planning form, results layout, map, weather chart, data-layer controls, and frontend–backend integration.
Georgios Malezoglou	Tour-planning logic and data processing	Will help define place filtering, weather-aware selection rules, stop ordering, duration estimates, and tests for planning behavior.
Abdullah	Testing and integration support	Will help test API and application behavior, check city-specific data integration, test unavailable-data scenarios, and coordinate technical documentation and integration checks.

6.2 AI tool usage disclosure

We used AI tools as part of our project preparation and development workflow, including exploring design options, supporting spec-driven development, reviewing and optimizing code, and considering architectural and implementation decisions. Our use of these tools supported the development process rather than replacing our own technical judgment.

We treat AI-generated suggestions as input rather than automatically accepting them. We review relevant outputs, consider design trade-offs, and check technical claims against API responses, available documentation, and implementation tests where applicable. We remain responsible for our final requirements, design decisions, code, and submitted work. This disclosure describes our collective workflow; any tool-specific attribution or additional AI use is documented according to the work actually performed by the team.

7 Conclusion

We designed City Tour Planner as a multi-city tour-planning application that combines weather, place, and traffic information with our own planning logic. We tested candidate APIs with Helsinki as our starting location and designed the application so that additional cities can be supported through suitable data providers.

Our proposed architecture separates the frontend, application workflow, planning logic, and provider integrations, with the Adapter pattern isolating external API formats, which keeps API details out of itinerary generation and makes the application easier to extend. Our Figma prototype demonstrates the planning workflow, loading feedback, results visualization, and handling of unavailable traffic information.

Our next implementation steps are to connect the agreed components, build the planning logic, integrate the frontend with the backend, and test the application against real API responses. We will use these tests to refine the itinerary and to ensure that the displayed information accurately reflects the available data.

References

[1] Helsinki Service Map API documentation. https://www.hel.fi/palvelukarttaws/restpages/ver4.html

[2] Service Map dataset information. https://avoindata.suomi.fi/data/en_GB/dataset/paakaupunkiseudun-palvelukartan-rest-rajapinta

[3] City Tour Planner Prototype, Figma. Open the Figma prototype